Strict aliasing in C++

Author: Damir Ljubic
email: damirlj@yahoo.com

Introduction

What is strict aliasing in C++?
Well, we can answer it by introducing two short definitions:

- Aliasing means that we have two (or more) expressions referring to the same memory location
- Strict aliasing means that compiler may assume that two (or more) pointers to unrelated types never alias. Standard brings certain rules on top of it – restrictions based on which the first definition holds

Accessing an object's stored value is UB (Undefined Behavior) if the **glvalue** (generalized value) of the accessing object's static type is not related with the stored object's dynamic type.
Precisely, the static type needs to be either:

- **The same as** dynamic type
- Similar to the dynamic type: where "similar" means differs only in **cv qualifiers** at some level
- Corresponding **sing/unsigned type** (arithmetic types)
- In case of the inheritance, **base class of the dynamic type**
- **Aggregates** and **unions** containing that type as the member

// NOK example

```
int a = 11; // glvalue representation of the object
auto p = reinterpret_cast<float*>(&a); // convertible to the static (float) type - but not the same or similar to it
*p = 5.3f; // UB!
```

// OK example

```
int a = -1; // glvalue representation of the object
auto p = reinterpret_cast<unsigned int*>(&a); // corresponding unsigned version
*p = 4;
```

- This also applies to the **byte-like types**, since any type can be read as an "raw bytes" array, while otherwise in not the case. We will see shortly – in custom allocator example

// OK example

```
static_assert(std::endian::native == std::endian::little);
int a = 11;
auto bytes = reinterpret_cast<std::byte*>(&a);
bytes[0] &= std::byte{0xFE};
assert (a == 10);
```

Custom allocators

For creating custom allocators, especially those that maintain internal memory storage on the stack, we use raw (uninitialized) bytes, to create an arbitrary type T.
Any type-aware allocation is two steps operation:

- Memory allocations (preallocated memory on the stack)
- Memory initialization (constructing the instance of T into memory): T becomes our **dynamic type** – no aliasing involving!

```
alignas(T) std::byte buff[SLOTS * sizeof(T)]; // internal memory storage on stack

auto mem = reinterpret_cast<T*>(&buff[slot * sizeof(T)]);
auto p = std::construct_at(mem, std::forward<Args>(args)…);
```

And now – when the memory is initialized for holding the concreate object of type T, we can safely access it.

A general note

> Forcibly casting the raw memory into some other type, with precondition that the type's footprint fits into the memory – calling the *reinterpret_cast*<> is not a priori UB. Accessing non-initialized memory, on the other hand, is undefined behavior